

Algorithms for Data Science

Exercise 2

See the cheatsheet for valuable remarks!

In our course: $\log$ always means $\log_2$.

Task 2.1: Oh-Notation

- (a) Someone tells you that an algorithm A has running time $O(n \log n)$ and an algorithm B has running time $O(n^2)$. Do you know which algorithm is faster?

Simplify the following running times. Explain the simplifications.

Difficult sample tasks explained by the tutor.

- (b) $\Theta(10 \cdot n^2 + 978 \cdot n + n \cdot n \cdot 2 \cdot n/1000)$
(c) $\Theta(1000(\log_5 n)^{1000} + \sqrt{n} \cdot 2000 + (11/10)^n)$
(d) $\Theta(4 \cdot n^{1/4} \cdot n^{0.25} + 2(2^n)^{2/n} + 50 \log_2(100n) + \log_3(9^n))$

Regular tasks for the students.

- (e) $\Theta(91n + n^2 \cdot 4n^2 + 3 \log n + 42n^3)$
(f) $\Theta(1000^{1000^{1000}} + 1/2 \cdot 1.0001^n + 40 \cdot n^{100})$
(g) $\Theta(5 \log(5n))$.
(h) $\Theta(5 \cdot 2^{5n+5})$.
(i) $\Theta(42m + k/100 + m \cdot 21 \cdot k + 2^{2 \log_2 n})$. *Sometimes there is more than one parameter.*

Bonus Tasks

Tasks for solving at home (not necessarily discussed in classes); relevant for short tests and the exam.

Task 2.2: Oh-Notation++

- (a) You are given the following running time functions (in Θ -notation). Order them by order of growth!

$$n! \quad (\log n)^2 \quad n^2 \quad \sqrt{n} \quad 2^{\sqrt{2} \log n} \quad 2^{2n} \quad n \cdot 2^n \quad \log(n^2) \quad 4^{\log n}$$

Prove your answers for the following questions.

- (b) Is $f(n) + g(n) = \Theta(\max(f(n), g(n)))$? (Assume that f and g are nonnegative functions.)
(c) Is $(n + 100)^5 = \Theta(n^5)$?
(d) Is $\log_3 n + \log_5 n = \Theta(\log_7 n)$?
(e) Is $300 \log(n^k k) + k^k + 2^{\sqrt{k} \log k} = O(n \log n + 2^{k \log k})$?

Task 2.3: Oh-Notation Definitions

- (a) In the lecture, we have seen running times depending on two parameters. For instance $O(kn)$. Propose a formal definition for $f(n_1, \dots, n_i) = O(g(n_1, \dots, n_i))$.
- (b) Prove that $f(n) = \Omega(g(n))$ holds if $g(n) = O(f(n))$ using the definition from the cheatsheet.

Task 2.4: Designing Algorithms: Airfield

Motivation: You want to build an airfield in a mountainous area. Therefore, you look for a longest possible plateau, that is, for a longest possible flat surface. An additional constraint is that the airfield has to be build along a fixed line of latitude since this is the direction in which the airplanes land or take off. To solve this problem, you first measure the height of the surface along the line of latitude at n equidistant sample points.

Now, given an integer array $h[1..n]$ with the measured heights (rounded to full meters), propose an efficient algorithm to find a longest plateau, that is, a longest consecutive subsequence $h[i..j]$ of the array where all the integers have the same value. The output should consist of a position i of the array where a longest plateau starts.

What is the running time of your algorithm?

Task 2.5: Designing Algorithms: Forest Age / Salary Database

- (a) *Motivation 1 (Forest):* Suppose that we have a database d with the age of all n trees of a forest (possibly obtained by some novel age-estimation technique using drones), where $d[i]$ is the age of the i -th tree. We also received m e-mails from different biologists, each one asking how many trees are there of a specific age. We store all the queries in a table q , where, for the j -th e-mail that we received, we store in $q[j]$ the age for which we have to compute the number of trees. Since the biologists are only interested in young trees, we know also that all values in q are smaller than some given small number k . (We assume that all the ages in q and d are rounded to full days.) Your task is to answer all the e-mails!

Motivation 2 (Salary): You maintain a database storing the yearly salary of all n persons in Poland. The database is implemented as an integer array $d[1..n]$ where $d[i]$ is the salary of the i -th person in Polish Złoty (PLN). Now, your boss gives you a small database containing the salary (in PLN) of some other m persons whose salary is not larger than k PLN, also implemented as an integer array $q[1..m]$. Your task is to compute, for each person of the database q , how many persons in the database d have exactly the same salary.

Input: arrays $d[1..n]$ and $q[1..m]$ of nonnegative integer values, and a parameter k with the guarantee that no value in q is larger than k .

Output: an array $res[1..m]$ where $res[i]$ is the number of indices $j \in \{1, \dots, n\}$ for which $q[i] = d[j]$.

Propose an algorithm solving this problem using only $O(1)$ additional memory space, that is, besides d , q , k and res , you are allowed to use only a constant number of variables.

What is the running time of your algorithm?

- (b) We consider the same problem as in (a). This time, propose an algorithm with running time $O(n + m + k)$ without any constraints regarding the memory space.

What is the extra memory used by your algorithm in Θ notation?

Compare your running time to (a) when assuming $m = \Theta(\sqrt{n})$ and $k = \Theta(m)$.

Hint: Use the counting technique.

Task 2.6: *Analyzing Algorithms: SomeSortingAlgo*

Algorithm 2: SomeSortingAlgo

Input: array $a[1..n]$ of integers**Output:** array a is sorted non-decreasingly

```
1  $i = 1$ 
2 while  $i \leq n - 1$  do
3    $minPos = i$ 
4    $j = i + 1$ 
5   while  $j < n$  do
6     if  $a[j + 1] < a[minPos]$  then
7        $minPos = j + 1$ 
8      $j++$ 
9   if  $minPos \neq i$  then
10    // exchanges the values at position  $i$  and  $minPos$  in the array  $a$ 
11     $Swap(a, i, minPos)$ 
12   $i++$ 
```

- (a) Write an algorithm for the function $Swap(a, i, \ell)$ that exchanges the values at position i and ℓ in the array a , that is, after calling $Swap(a, i, \ell)$, $a[i]$ contains the former value at $a[\ell]$, and $a[\ell]$ contains the former value at $a[i]$.
- (b) Give the best case and the worst case running times in Θ -notation of **SomeSortingAlgo**.
- (c) The algorithm has a mistake. Give a shortest possible input array for which the algorithm computes a wrong output. Describe the mistake and fix it by changing the code as little as possible.
- (d) Prove that in lines 3 to 8, the algorithm that you repaired in task (c) finds a position $minPos$ of a smallest element in $a[i..n]$ (that is, $a[minPos]$ is this smallest element). For your proof, come up with a helpful loop invariant.
- (e) Prove the correctness of the whole algorithm **SomeSortingAlgo** via the following invariant (defined with respect to line 2):

The subarray $a[1..i]$ is sorted non-decreasingly, and there is no element in the subarray $a[1..i - 1]$ that is larger than some element in $a[i..n]$. Furthermore $a[1..n]$ contains all the elements that it contained in the beginning. (For $i = 1$, we define $a[1..i - 1] = a[1..0]$ as an empty array.)

Recall that you have to prove that the invariant holds (i) in the beginning (when $i = 1$), and (ii) that it holds for each $i > 1$ using the assumption that it holds for $i - 1$ (equivalently: that it holds for $i + 1$ assuming that it holds for $i \geq 1$). Finally, use the invariant to conclude that at the end the whole input array is sorted.

Task 2.7: Analyzing Algorithms: StrangeAlgo

Algorithm 4: StrangeAlgo

Input: nonnegative integers n and k

Output: ?

```
1 StrangeAlgo( $n, k$ )
2   if  $k == 0$  then
3     return 0
4   if  $k == 1$  then
5     return 1
6    $a[1..1000] = \text{new array}$ 
7    $i = 1$ 
8   while  $i \leq 1000$  do
9      $a[i] = i$ 
10     $i++$ 
11   while  $i \leq n$  do
12      $i++$ 
13    $ret = \text{StrangeAlgo}(n, k - 2)$ 
14   return  $ret$ 
```

- (a) What is the output of `StrangeAlgo(1000000000000000, 8941307)`?
- (b) What is the running time of `StrangeAlgo( $n, k$ )` in Θ -notation in dependence of n and k ?
- (c) What is the memory complexity of `StrangeAlgo( $n, k$ )` in Θ -notation in dependence of n and k ?
- (d) Modify `StrangeAlgo( $n, k$ )` to run in time $\Omega(n^k)$.

Advanced Tasks

Challenging tasks for motivated students, but beyond the scope of this course. Their difficulty level is comparable to that of standard computer science courses.

Task 2.8: Cyclic Rotation

Given an array $a[1..n]$ and an integer $k \in [1, n]$, rotate the elements in a by k elements to the left. In other words, at the end when your program stops, for every $1 \leq i \leq n$, $a[i]$ should contain the value that initially was at position $a[(i + k) \% n]$.

Find the following two solutions, both solutions with optimal asymptotic running time and $\Theta(1)$ extra space complexity!

- (a) In the first solution, minimize the number of times you access the array; for this solution, you are allowed to call the function `gcd( $i, j$ )` that returns in time $\Theta(\log^2 i + \log^2 j)$ the greatest common divisor of i and j .
- (b) For the second solution, the only thing you cannot access the array directly but you are allowed to call the function `Reverse( $a, i, j$ )` that reverses the element order in the subarray $a[i..j]$ (in time $\Theta(j - i)$).